

使用 Gemini Pro 建構採用多模態 RAG 的問答應用程式

來源：<https://codelabs.developers.google.com/multimodal-rag-gemini?hl=zh-tw>

步驟數：12

在這個程式碼研究室中，您將瞭解如何使用 Gemini Pro 建構多模態問題回答系統。

目錄

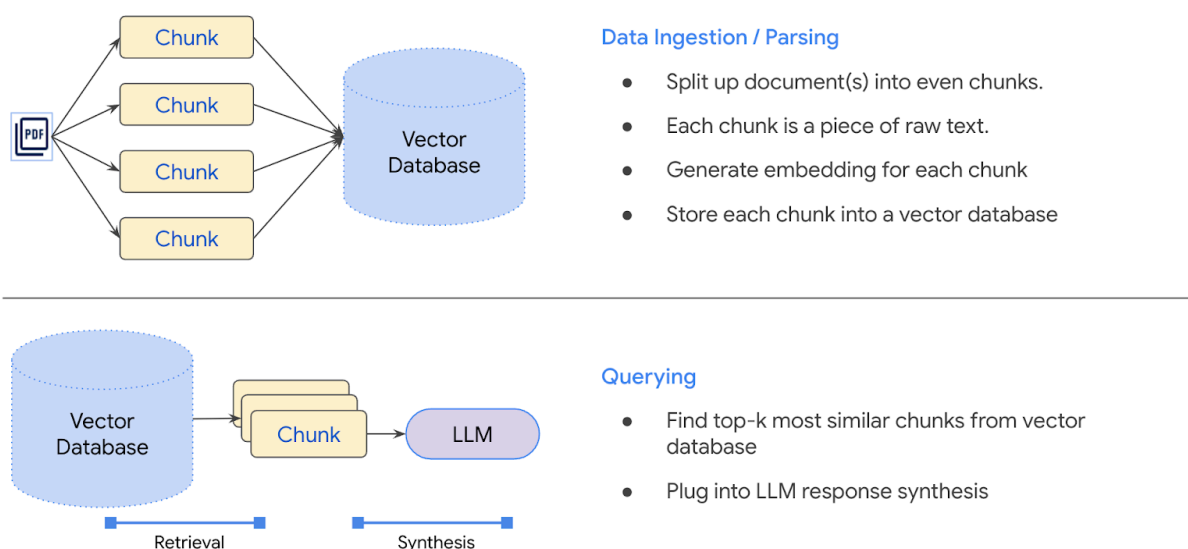
1. 簡介
2. 事前準備
3. 建構多模態 RAG
4. 步驟 1：安裝及匯入依附元件
5. 步驟 2：準備及載入資料
6. 步驟 3：產生文字摘要
7. 步驟 4：生成圖片摘要
8. 步驟 5：建構多向量擷取功能
9. 步驟 6：建構多模態 RAG
10. 步驟 7：測試查詢
11. 清除所用資源
12. 恭喜

1. 簡介

什麼是 RAG

檢索增強生成 (RAG) 技術結合大型語言模型 (LLM) 的強大功能，以及從外部知識來源檢索相關資訊的能力。也就是說，LLM 不僅依賴內部訓練資料，還能在生成回覆時存取並納入最新特定資訊。

RAG workflow for building a QA System



RAG 越來越受歡迎的原因如下：

- **提高準確度和關聯性：**RAG 可讓 LLM 根據從外部來源擷取的事實資訊，提供更準確且相關的回覆。這項功能在需要最新知識的場合特別實用，例如回答有關時事的問題，或提供特定主題的資訊。
- **減少幻覺：**LLM 有時會生成看似合理，但其實錯誤或無意義的回覆。RAG 會根據外部來源驗證生成的資訊，藉此減輕這個問題。
- **適應性更強：**RAG 可讓 LLM 更能適應不同領域和工作。只要運用不同的知識來源，就能輕鬆自訂 LLM，提供各種主題的資訊。
- **提升使用者體驗：**RAG 可提供更豐富、可靠且相關的回覆，進而提升整體使用者體驗。

為何要使用多模態

在現今這個資訊爆炸的時代，文件通常會結合文字和圖片，以全面傳達資訊。不過，大多數檢索增強生成 (RAG) 系統都會忽略圖片中隱藏的寶貴洞察資料。隨著多模態大型語言模型 (LLM) 越來越受歡迎，我們必須探索如何在 RAG 中同時運用視覺內容和文字，進一步瞭解資訊環境。

多模態 RAG 的兩種選項

- **多模態嵌入 -** 多模態嵌入模型會根據您提供的輸入內容 (可包含圖片、文字和影片資料的組合)，生成 1408 維度的向量*。圖片嵌入向量和文字嵌入向量位於相同的語意空間，且維度相同。因此，這些向量可互換使用，適用於以文字搜尋圖片或以圖片搜尋影片等用途。請參閱這份[示範](#)。

1. 使用多模態嵌入模型嵌入文字和圖片
2. 使用相似度搜尋擷取兩者
3. 將擷取的原始圖片和文字區塊傳遞至 **多模態 LLM**，以合成答案

- 文字嵌入 -

1. 使用多模態 LLM 生成圖像的文字摘要
2. 嵌入及擷取文字
3. 將文字區塊傳遞至 LLM，以合成答案

什麼是多向量檢索器

多向量檢索會使用文件各節的摘要，擷取原始內容來合成答案。這項功能可提升 RAG 的品質，特別是處理表格、圖表等需要大量運算資源的工作。詳情請參閱 [Langchain 的網誌](#)。

建構項目

用途：使用 Gemini Pro 開發問答系統

假設您有包含複雜圖表或圖解的文件，其中含有大量資訊。您想擷取這項資料，以回答問題或查詢。

在本程式碼研究室中，您將執行下列操作：

- 使用 [LangChain](#) 載入資料 `document_loaders`
 - 使用 Google 的 `gemini-pro` 模型生成文字摘要
 - 使用 Google 的 `gemini-pro-vision` 模型生成圖片摘要
 - 使用 Google 的 `textembedding-gecko` 模型，以 Croma Db 做為向量儲存庫，建立多向量擷取功能
 - 開發用於問題回答的多模態 RAG 鏈
-

2. 事前準備

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
 2. 確保您的 Google Cloud 雲端專案有啟用計費服務。瞭解如何[檢查專案是否已啟用計費功能](#)。
 3. 在 Vertex AI 資訊主頁中啟用[所有建議的 API](#)
 4. 開啟 [Colab 筆記本](#)，並登入與目前有效 Google Cloud 帳戶相同的帳戶。
-

步驟 3 · 約 0 分鐘

3. 建構多模態 RAG

本程式碼實驗室使用 [Vertex AI SDK for Python](#) 和 [Langchain](#)，示範如何在 Google Cloud 中實作[這裡](#)所述的「選項 2」。

如需完整程式碼，請參閱參考的[存放區](#)中的「[Multi-modal RAG with Google Cloud](#)」檔案。

步驟 4 · 約 0 分鐘

4. 步驟 1：安裝及匯入依附元件

```
!pip install -U --quiet langchain langchain_community chromadb langchain-google-vertexai
!pip install --quiet "unstructured[all-docs]" pypdf pillow pydantic lxml pillow matplotlib
chromadb tiktoken
```

輸入專案 ID 並完成驗證

```
#TODO : ENter project and location
PROJECT_ID = ""
REGION = "us-central1"

from google.colab import auth
auth.authenticate_user()
```

初始化 Vertex AI 平台

```
import vertexai
vertexai.init(project = PROJECT_ID , location = REGION)
```

步驟 5 · 約 0 分鐘

5. 步驟 2：準備及載入資料

我們使用 ZIP 檔案，內含從[這篇](#)網誌文章擷取的圖片和 PDF 子集。如要瞭解完整流程，請使用原始範例。

請先下載資料

```
import logging
import zipfile
import requests

logging.basicConfig(level=logging.INFO)

data_url = "https://storage.googleapis.com/benchmarks-artifacts/langchain-docs-benchmarking/cj.zip"
result = requests.get(data_url)
filename = "cj.zip"
with open(filename, "wb") as file:
    file.write(result.content)

with zipfile.ZipFile(filename, "r") as zip_ref:
    zip_ref.extractall()
```

從文件載入文字內容

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("./cj/cj.pdf")
docs = loader.load()
tables = []
texts = [d.page_content for d in docs]
```

檢查第一頁的內容

```
texts[0]
```

您應該會看到輸出內容

```
'11/14/23, 8:35 PM Clouded Judgement 11.10.23 - by Jamin Ball\nhttps://cloudedjudgement.substack.com/p/clouded-judgement-111023 1/21Clouded Judgement 11.10.23 - OpenAI\nUpdates + Datadog Gives the All-Clear?\nJAMIN BALL\nNOV 10, 2023\n2 Share\nEvery week I'll provide updates on the latest trends in cloud software companies. Follow along to\nstay up to date!\nOpen AI U pdates\nOpen AI had their big developer day this week, and I wanted to call out two key announcements\n(and trends): increasing context windows and decreasing costs.\nWhen I think about the monetization of AI (and which "layers" monetize\nrst) I've always\nthought it would follow the below order, with each layer lagging the one that comes before it.\n1. Raw silicon (chips like Nvidia bought i\nn large quantities to build out infra to service\nupcoming demand).\n2. Model providers (OpenAI, Anthropic, etc as companies start building out AI).\n35\nType your email... Subscribe'
```

文件總頁數

```
len(texts)
```

預期輸出內容為

步驟 6 · 約 0 分鐘

6. 步驟 3：產生文字摘要

請先匯入必要的程式庫

```
from langchain_google_vertexai import VertexAI , ChatVertexAI , VertexAIEmbeddings
from langchain.prompts import PromptTemplate
from langchain_core.messages import AIMessage
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
```

取得文字摘要

```

# Generate summaries of text elements
def generate_text_summaries(texts, tables, summarize_texts=False):
    """
    Summarize text elements
    texts: List of str
    tables: List of str
    summarize_texts: Bool to summarize texts
    """

    # Prompt
    prompt_text = """You are an assistant tasked with summarizing tables and text for
retrieval. \
These summaries will be embedded and used to retrieve the raw text or table elements. \
Give a concise summary of the table or text that is well optimized for retrieval. Table or
text: {element} """
    prompt = PromptTemplate.from_template(prompt_text)
    empty_response = RunnableLambda(
        lambda x: AIMessage(content="Error processing document")
    )
    # Text summary chain
    model = VertexAI(
        temperature=0, model_name="gemini-pro", max_output_tokens=1024
    ).with_fallbacks([empty_response])
    summarize_chain = {"element": lambda x: x} | prompt | model | StrOutputParser()

    # Initialize empty summaries
    text_summaries = []
    table_summaries = []

    # Apply to text if texts are provided and summarization is requested
    if texts and summarize_texts:
        text_summaries = summarize_chain.batch(texts, {"max_concurrency": 1})
    elif texts:
        text_summaries = texts

    # Apply to tables if tables are provided
    if tables:
        table_summaries = summarize_chain.batch(tables, {"max_concurrency": 1})

    return text_summaries, table_summaries

# Get text summaries
text_summaries, table_summaries = generate_text_summaries(
    texts, tables, summarize_texts=True
)

text_summaries[0]

```

預期輸出內容為

Summary of "Clouded Judgement 11.10.23"\n\nThis article, published on November 10, 2023, by Jamin Ball, discusses the latest trends in cloud software companies. \n\n**Key points:**\n\n* OpenAI had a developer day with two key announcements: increasing context windows and decreasing costs.\n* The author believes that AI monetization will follow a specific order, with raw silicon being monetized first, followed by model providers, and finally applications.\n* Datadog is mentioned as giving the "all-clear" for some unspecified event.\n\n**Keywords:**\n\n* OpenAI\n* cloud software\n* AI monetization\n* Datadog\n\n**This summary is optimized for retrieval by including the key points, keywords, and relevant information from the article.** \n'

步驟 7 · 約 0 分鐘

7. 步驟 4：生成圖片摘要

請先匯入必要的程式庫

```
import base64
import os

from langchain_core.messages import HumanMessage
```

生成圖片摘要

```

def encode_image(image_path):
    """Getting the base64 string"""
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

def image_summarize(img_base64, prompt):
    """Make image summary"""
    model = ChatVertexAI(model_name="gemini-pro-vision", max_output_tokens=1024)

    msg = model(
        [
            HumanMessage(
                content=[
                    {"type": "text", "text": prompt},
                    {
                        "type": "image_url",
                        "image_url": {"url": f"data:image/jpeg;base64,{img_base64}"},
                    },
                ],
            )
        ]
    )
    return msg.content

def generate_img_summaries(path):
    """
    Generate summaries and base64 encoded strings for images
    path: Path to list of .jpg files extracted by Unstructured
    """

    # Store base64 encoded images
    img_base64_list = []

    # Store image summaries
    image_summaries = []

    # Prompt
    prompt = """You are an assistant tasked with summarizing images for retrieval. \
These summaries will be embedded and used to retrieve the raw image. \
Give a concise summary of the image that is well optimized for retrieval."""

    # Apply to images
    for img_file in sorted(os.listdir(path)):
        if img_file.endswith(".jpg"):
            img_path = os.path.join(path, img_file)
            base64_image = encode_image(img_path)
            img_base64_list.append(base64_image)
            image_summaries.append(image_summarize(base64_image, prompt))

    return img_base64_list, image_summaries

```

```
# Image summaries
img_base64_list, image_summaries = generate_img_summaries("./cj")

len(img_base64_list)

len(image_summaries)

image_summaries[0]
```

您應該會看到類似以下的輸出內容

```
' The image shows a line chart of EV/NTM revenue multiples over time. The three lines represent different growth rates. The blue line is the highest growth rate, the red line is the median growth rate, and the orange line is the lowest growth rate. The chart shows that EV/NTM revenue multiples have been increasing over time, with the highest growth rate stocks leading the way.'
```

步驟 8 · 約 0 分鐘

8. 步驟 5：建構多向量擷取功能

現在來生成文字和圖片摘要，並儲存至 ChromaDB 向量儲存庫。

匯入必要程式庫

```
import uuid
from langchain.retrievers.multi_vector import MultiVectorRetriever
from langchain.storage import InMemoryStore
from langchain_community.vectorstores import Chroma
from langchain_core.documents import Document
```

建立多向量檢索


```

def create_multi_vector_retriever(
    vectorstore, text_summaries, texts, table_summaries, tables, image_summaries, images
):
    """
    Create retriever that indexes summaries, but returns raw images or texts
    """

    # Initialize the storage layer
    store = InMemoryStore()
    id_key = "doc_id"

    # Create the multi-vector retriever
    retriever = MultiVectorRetriever(
        vectorstore=vectorstore,
        docstore=store,
        id_key=id_key,
    )

    # Helper function to add documents to the vectorstore and docstore
    def add_documents(retriever, doc_summaries, doc_contents):
        doc_ids = [str(uuid.uuid4()) for _ in doc_contents]
        summary_docs = [
            Document(page_content=s, metadata={id_key: doc_ids[i]})
            for i, s in enumerate(doc_summaries)
        ]
        retriever.vectorstore.add_documents(summary_docs)
        retriever.docstore.mset(list(zip(doc_ids, doc_contents)))

    # Add texts, tables, and images
    # Check that text_summaries is not empty before adding
    if text_summaries:
        add_documents(retriever, text_summaries, texts)
    # Check that table_summaries is not empty before adding
    if table_summaries:
        add_documents(retriever, table_summaries, tables)
    # Check that image_summaries is not empty before adding
    if image_summaries:
        add_documents(retriever, image_summaries, images)

    return retriever

# The vectorstore to use to index the summaries
vectorstore = Chroma(
    collection_name="mm_rag_cj_blog",
    embedding_function=VertexAIEmbeddings(model_name="textembedding-gecko@latest"),
)

# Create retriever
retriever_multi_vector_img = create_multi_vector_retriever(
    vectorstore,
    text_summaries,

```

```
texts,  
table_summaries,  
tables,  
image_summaries,  
img_base64_list,  
)
```

步驟 9 · 約 0 分鐘

9. 步驟 6：建構多模態 RAG

1. 定義公用函式

```

import io
import re

from IPython.display import HTML, display
from langchain_core.runnables import RunnableLambda, RunnablePassthrough
from PIL import Image

def plt_img_base64(img_base64):
    """Disply base64 encoded string as image"""
    # Create an HTML img tag with the base64 string as the source
    image_html = f''
    # Display the image by rendering the HTML
    display(HTML(image_html))

def looks_like_base64(sb):
    """Check if the string looks like base64"""
    return re.match("^[A-Za-z0-9+/]+=]{0,2}$", sb) is not None

def is_image_data(b64data):
    """
    Check if the base64 data is an image by looking at the start of the data
    """
    image_signatures = {
        b"\xFF\xD8\xFF": "jpg",
        b"\x89\x50\x4E\x47\x0D\x0A\x1A\x0A": "png",
        b"\x47\x49\x46\x38": "gif",
        b"\x52\x49\x46\x46": "webp",
    }
    try:
        header = base64.b64decode(b64data)[:8] # Decode and get the first 8 bytes
        for sig, format in image_signatures.items():
            if header.startswith(sig):
                return True
        return False
    except Exception:
        return False

def resize_base64_image(base64_string, size=(128, 128)):
    """
    Resize an image encoded as a Base64 string
    """
    # Decode the Base64 string
    img_data = base64.b64decode(base64_string)
    img = Image.open(io.BytesIO(img_data))

    # Resize the image
    resized_img = img.resize(size, Image.LANCZOS)

```

```

# Save the resized image to a bytes buffer
buffered = io.BytesIO()
resized_img.save(buffered, format=img.format)

# Encode the resized image to Base64
return base64.b64encode(buffered.getvalue()).decode("utf-8")

def split_image_text_types(docs):
    """
    Split base64-encoded images and texts
    """
    b64_images = []
    texts = []
    for doc in docs:
        # Check if the document is of type Document and extract page_content if so
        if isinstance(doc, Document):
            doc = doc.page_content
            if looks_like_base64(doc) and is_image_data(doc):
                doc = resize_base64_image(doc, size=(1300, 600))
                b64_images.append(doc)
            else:
                texts.append(doc)
    if len(b64_images) > 0:
        return {"images": b64_images[:1], "texts": []}
    return {"images": b64_images, "texts": texts}

```

2. 定義特定領域的圖像提示

```

def img_prompt_func(data_dict):
    """
    Join the context into a single string
    """
    formatted_texts = "\n".join(data_dict["context"]["texts"])
    messages = []

    # Adding the text for analysis
    text_message = {
        "type": "text",
        "text": (
            "You are financial analyst tasking with providing investment advice.\n"
            "You will be given a mixed of text, tables, and image(s) usually of charts or\n"
            "graphs.\n"
            "Use this information to provide investment advice related to the user question.\n"
            "\n"
            f"User-provided question: {data_dict['question']}\n\n"
            "Text and / or tables:\n"
            f"{formatted_texts}"
        ),
    }
    messages.append(text_message)
    # Adding image(s) to the messages if present
    if data_dict["context"]["images"]:
        for image in data_dict["context"]["images"]:
            image_message = {
                "type": "image_url",
                "image_url": {"url": f"data:image/jpeg;base64,{image}"},
            }
            messages.append(image_message)
    return [HumanMessage(content=messages)]

```

3. 定義多模態 RAG 鏈

```
def multi_modal_rag_chain(retriever):
    """
    Multi-modal RAG chain
    """

    # Multi-modal LLM
    model = ChatVertexAI(
        temperature=0, model_name="gemini-pro-vision", max_output_tokens=1024
    )

    # RAG pipeline
    chain = (
        {
            "context": retriever | RunnableLambda(split_image_text_types),
            "question": RunnablePassthrough(),
        }
        | RunnableLambda(img_prompt_func)
        | model
        | StrOutputParser()
    )

    return chain

# Create RAG chain
chain_multimodal_rag = multi_modal_rag_chain(retriever_multi_vector_img)
```

步驟 10 · 約 0 分鐘

10. 步驟 7：測試查詢

1. 擷取相關文件

```
query = "What are the EV / NTM and NTM rev growth for MongoDB, Cloudflare, and Datadog?"
docs = retriever_multi_vector_img.get_relevant_documents(query, limit=1)

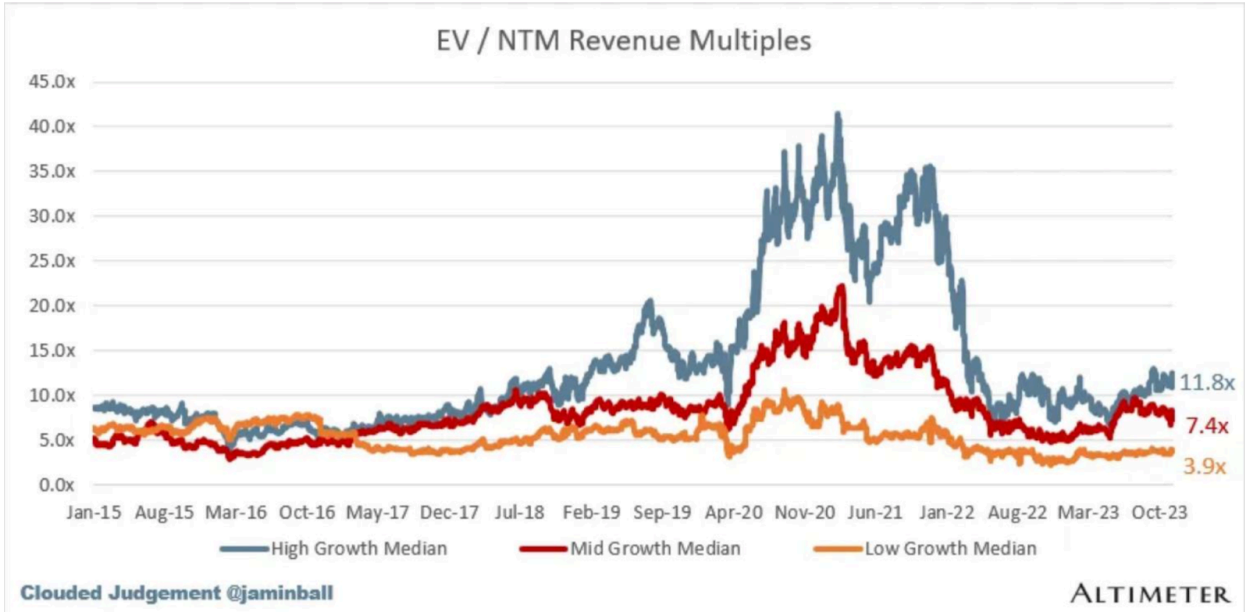
# We get relevant docs
len(docs)

docs
```

You may get similar output

[illegible]

```
plt.imshow(img_base64(docs[3]))
```



2. 對相同查詢執行 RAG

```
result = chain_multimodal_rag.invoke(query)

from IPython.display import Markdown as md
md(result)
```

輸出範例 (執行程式碼時可能有所不同)

Company	EV/NTM Rev	NTM Rev	Growth
MongoDB	14.6x		17%
Cloudflare	13.4x		28%
Datadog	13.1x		19%

11. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本程式碼研究室所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

步驟 12 · 約 0 分鐘

12. 恭喜

恭喜！您已成功使用 Gemini 開發多模態 RAG。
